PROJECT REPORT

RESUME ANALYZER AND JOB RECOMMENDATION SYSTEM

Bachelor of Computer Applications (BCA)

Submitted by: [Student Name]
Register Number: [Register Number]
College: [College Name]
Project Guide: [Guide Name]
Academic Year: [Academic Year]

Certificate

This is to certify that the project titled “Resume Analyzer and Job Recommendation System” is a bonafide work carried out by [Student Name] under the guidance of [Guide Name] in partial fulfilment of the requirements for the Bachelor of Computer Applications degree. Replace this sample text with your institution's required format.

Declaration

I declare that this project report represents work completed by me for academic purposes. Third-party libraries and references used have been acknowledged. Adapt this declaration to your institution's policy.

Acknowledgement

I would like to thank my project guide, department faculty, and institution for their support and feedback during this project.

Abstract

The Resume Analyzer and Job Recommendation System is a web application that extracts text from PDF/DOCX resumes, detects known skills, and compares resume content with sample job descriptions. It uses Python, Flask, SQLite, PyMuPDF, python-docx, and scikit-learn. Users can see matching skills, possible skill gaps, ranked sample jobs, saved jobs, and analysis history. Scores are similarity estimates and are not predictions of hiring success.

1. Introduction

Students may find it difficult to compare their resumes with job requirements and identify missing skills. This project offers an automated first-pass comparison using skill extraction and text similarity.

1.1 Objectives

• Provide registration and login.
• Support PDF and DOCX uploads.
• Extract text and identify known skills.
• Compare resumes with job descriptions.
• Show match scores and skill gaps.
• Save jobs and analysis history.

2. Problem Statement

Candidates often compare resumes and job descriptions manually. This process can be repetitive and may not clearly reveal which skills are missing. The proposed system organizes this process in a single web interface.

3. Proposed System

The system uses Flask routes to manage accounts, uploads, jobs, and dashboard pages. The analyzer extracts document text, detects skills using a configurable vocabulary, and computes a score using skill coverage and TF-IDF cosine similarity.

4. Technology Stack

Python 3.11+, Flask, SQLite, HTML, CSS, Bootstrap, JavaScript, PyMuPDF, python-docx, scikit-learn, pytest, and VS Code.

5. System Modules

Authentication; resume upload and extraction; skill extraction; job listing/search; resume-job matching; skill-gap reporting; saved jobs; analysis history.

6. Algorithm

1. Upload and validate a PDF/DOCX file.
2. Extract document text.
3. Normalize text and detect vocabulary skills.
4. Detect known skills in each job description.
5. Compute skill coverage = matched job skills / detected job skills.
6. Compute TF-IDF cosine similarity.
7. Score = 75% skill coverage + 25% text similarity.
8. Rank roles and display matched/missing skills.

7. Database Design

The SQLite database contains users, resumes, skills, resume_skills, jobs, job_skills, analysis_results, and saved_jobs. The detailed column list and relationships are provided in docs/database_design.md.

8. Testing

Automated analyzer tests are included in tests/test_analyzer.py. Manual tests should cover account creation, login, invalid uploads, PDF/DOCX parsing, job search, matching, saved jobs, and analysis history. Record actual results in docs/testing.md.

9. Limitations

The vocabulary is limited, scanned PDFs require OCR, job records are sample listings rather than live vacancies, and TF-IDF is a basic text-similarity method. The score is not a hiring probability.

10. Future Enhancements

Add an admin panel, OCR, skill synonyms, semantic embeddings, downloadable reports, live job integrations where permitted, MySQL deployment, CSRF protection, rate limiting, file scanning, and privacy/retention controls.

11. Conclusion

The project demonstrates a practical integration of Python web development, relational database design, document processing, and basic NLP. It helps users compare resumes with sample job descriptions and identify skills to review.

References

Flask: https://flask.palletsprojects.com/
Python: https://docs.python.org/3/
scikit-learn: https://scikit-learn.org/stable/
PyMuPDF: https://pymupdf.readthedocs.io/
python-docx: https://python-docx.readthedocs.io/
SQLite: https://www.sqlite.org/docs.html